HW 05 - Legos

Max Asmar

Figure 1: Photo by Daniel Cheung on Unsplash

This week we'll do some data gymnastics to refresh and review what we learned over the past few weeks using (simulated) data from Lego sales in 2018 for a sample of customers who bought Legos in the US.

Getting started

Open the `hw-05-legos.Rmd` file in your RStudio Notebook on JupyterHub. Knit the document to make sure it compiles without errors.

Warm up

Before we introduce the data, let's warm up with some simple exercises. Update the YAML of your R Markdown file with your information, knit, commit, and push your changes. Make sure to commit with a meaningful commit message. Then, go to your repo on GitHub and confirm that your changes are visible in your Rmd **and** md files. If anything is missing, commit and push again.

Packages

We'll use the **tidyverse** package for much of the data wrangling and visualisation and the data lives in the **dsbox** package. These packages are already installed for you. You can load them by running the following in your Console:

```
library(tidyverse)
library(dsbox)
```

Data

The data can be found in the **dsbox** package, and it's called `lego_sales`. Since the dataset is distributed with the package, we don't need to load it separately; it becomes available to us when we load the package. You can find out more about the dataset by inspecting its documentation, which you can access by running `?lego_sales` in the Console or using the Help menu in RStudio to search for `lego_sales`. You can also find this information here.

Explore the Data

Before you start the exercises, take a few minutes to explore the `lego_sales` dataset. Run the following code in your Console (not in your Rmd file) to get familiar with the data:

```
# View the structure of the data
glimpse(lego_sales)

# Look at the first few rows
head(lego_sales)

# Get summary statistics
summary(lego_sales)

# View the full dataset in a spreadsheet-style viewer
# View(lego_sales)
```

Questions to consider as you explore: - How many rows (observations) are in the dataset?

There are 620 observations in the dataset. I found this by using the glimpse() function on the dataset.

- What variables are included?

The variables for each observation are first_name, last_name, age, phone_number, set_id, number, theme, subtheme, year, name, pieces, us_price, image_url, and quantity. I found this using the glimpse() function on the dataset.

- What types of data are in each variable (numeric, character, etc.)?

The numeric variables are age, set_id, number, year, pieces, us_price, and quantity. The string variables are first_name, last_name, phone_number, theme, subtheme, name, and image_url. I found this by using the glimpse() function on the dataset.

- Are there any missing values (NAs)?

Yes, there are many missing values in the dataset. I found this by using the summary() function on the dataset.

- What is the range of ages? Prices? Quantities?

Using the summary() function on the dataset, I found that the range of ages is from 16 to 68 years inclusive. I also determined that the range of prices is from 3.99 to 349.99 dollars, inclusive. The range of quantities is from 1 to 5, inclusive.

Understanding your data before you start analyzing it is a crucial first step in any data science project!

Helpful Functions

Here's a quick reference of functions you'll need for this assignment. You've used all of these in previous assignments, but this is a handy reminder of what each does:

Function	Purpose	Example
`count()`	Count occurrences of values	`data %>% count(variable, sort = TRUE)`
`filter()`	Keep rows that meet a condition	`data %>% filter(age > 25)`
`group_by()`	Group data by a variable	`data %>% group_by(category)`
`summarise()`	Calculate summary statistics	`summarise(total = sum(quantity))`
`mutate()`	Create or modify variables	`mutate(new_var = var1 * var2)`
`case_when()`	Conditional logic (like if-else)	`case_when(age <= 18 ~ "child", age > 18 ~ "adult")`
`str_sub()`	Extract part of a text string	`str_sub(phone, 1, 3)` extracts first 3 characters
`arrange()`	Sort rows	`arrange(desc(total))` sorts high to low
`slice()`	Select specific rows by position	`slice(1:3)` gets first 3 rows

Remember: Use the pipe operator `%>%` to chain these functions together!

Exercises

Answer the following questions using pipelines. For each question, state your answer in a sentence, e.g. "In this sample, the first three common names of purchasers are …". Note that the answers to all questions are within the context of this particular sample of sales, i.e. you shouldn't make inferences about the population of all Lego sales based on this sample.

Getting Started with Pipelines

For the first few questions, we'll provide some code structure to help you get started. Fill in the blanks (indicated by `_____`) with the appropriate variable names or arguments.

Understanding Your Output

When you run your code, your output should be displayed in a table format. Here's an example of what the output for a `count()` function might look like:

```
## # A tibble: 3 x 2
##   first_name     n
##   <chr>      <dbl>
## 1 Michael       45
## 2 Jennifer      42
## 3 Matthew       38
```

The `n` column shows how many times each name appears in the dataset. Your job is to interpret this output and write your answer in a complete sentence.

1. What are the three most common first names of purchasers?

Hints: What variable contains the first names? Do you want the results sorted in ascending or descending order?

```
lego_sales %>%
  count(first_name, sort = TRUE) %>%
  slice(1:3)
```

```
## # A tibble: 3 x 2
##   first_name     n
##   <chr>      <int>
## 1 Jackson       13
## 2 Jacob         11
## 3 Joseph        11
```

Write your answer here: In this sample, the three most common first names of purchasers are Jackson, Jacob, and Joseph. Each first name has 13, 11, and 11 observations respectively.

2. What are the three most common themes of Lego sets purchased?

```
lego_sales %>%
  count(theme, sort = TRUE) %>%
  slice(1:3)
```

```
## # A tibble: 3 x 2
##   theme            n
##   <chr>        <int>
## 1 Star Wars       75
## 2 Nexo Knights    64
## 3 Gear            55
```

Your output should show three rows with a theme name and count (n).

Write your answer here: In this sample, the three most common themes of Lego sets purchased are Star Wars, Nexo Knights, and Gear themes. Each theme has 75, 64, and 55 observations respectively.

3. Among the most common theme of Lego sets purchased, what is the most common subtheme?

```
lego_sales %>%
  filter(theme == "Star Wars") %>%
  count(subtheme, sort = TRUE) %>%
  slice(1:3)
```

```
## # A tibble: 3 x 2
##   subtheme               n
##   <chr>              <int>
## 1 The Force Awakens     15
## 2 Buildable Figures     11
## 3 Episode V             10
```

Among the most common theme of Lego sets purchased (Star Wars), the most common subtheme is The Force Awakens, with 15 observations.

Knit, *commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards.*

4. Create a new variable called `age_group` and group the ages into the following categories: “18 and under”, “19 - 25”, “26 - 35”, “36 - 50”, “51 and over”.

Hint: Use the `case_when()` function within `mutate()`. The structure is: mutate(age_group = case_when(age <= 18 ~ “18 and under”, age >= 19 & age <= 25 ~ “19 - 25”, …)) Each condition is tested in order. The ~ separates the condition from the label.

```
# Your code here
lego_sales <- lego_sales %>%
  mutate(age_group = case_when(
    age <= 18 ~ "18 and under",
    age >= 19 & age <= 25 ~ "19 - 25",
    age >= 26 & age <= 35 ~ "26 - 35",
    age >= 36 & age <= 50 ~ "36 - 50",
    age >= 51 ~ "51 and over"
    ))
```

After creating the variable, verify it worked by viewing the first few rows with `select()` and `head()`, and count how many observations fall into each age group.

```
lego_sales %>%
  select(age, age_group) %>%
  head()
```

```
## # A tibble: 6 x 2
##     age age_group
##   <dbl> <chr>
## 1    24 19 - 25
## 2    35 26 - 35
## 3    35 26 - 35
## 4    41 36 - 50
## 5    41 36 - 50
## 6    41 36 - 50
```

```
lego_sales %>%
  count(age_group, sort = TRUE)
```

```
## # A tibble: 5 x 2
##   age_group        n
##   <chr>        <int>
## 1 36 - 50        216
## 2 26 - 35        183
## 3 19 - 25        129
## 4 51 and over     62
## 5 18 and under    30
```

Write your answer here: I created the age_group variable. The distribution shows that most of the customers fall into the 36 - 50 age range, with a total of 216 observations. This makes up around 1/3 of the dataset. The least popular age group is 18 and under, with only 30 observations in the dataset. There is over a 50% chance that a lego customer in the dataset is in between 26 and 50 years old. I determined this by adding the number of observations for this group (216+183), out of the total observations in the dataset(620).

5. Which age group has purchased the highest number of Lego sets.

Important: Remember that `quantity` tells you how many sets were purchased. If someone bought quantity = 2, that counts as 2 Lego sets, not 1 purchase.

```
# Your code here
lego_sales %>%
  group_by(age_group) %>%
  summarise(total_quantity = sum(quantity)) %>%
  arrange(desc(total_quantity))
```

```
## # A tibble: 5 x 2
##   age_group    total_quantity
##   <chr>                 <dbl>
```

```
## 1 36 - 50                  313
## 2 26 - 35                  267
## 3 19 - 25                  174
## 4 51 and over               92
## 5 18 and under              45
```

Think about: What do you need to sum? What do you need to group by?

Write your answer here:

The age group that has purchased the highest number of Lego sets is the age range from 36-50. In the dataset, they have purchased 313 sets in total.

6. Which age group has spent the most money on Legos?

Hint: For each row, total spending = quantity × price. You'll need to create this calculation, then sum by age group.

```
lego_sales %>%
  group_by(age_group) %>%
  summarise(total_price = sum(us_price*quantity)) %>%
  arrange(desc(total_price))
```

```
## # A tibble: 5 x 2
##   age_group    total_price
##   <chr>              <dbl>
## 1 36 - 50            9533.
## 2 26 - 35            7576.
## 3 19 - 25            4939.
## 4 51 and over        2475.
## 5 18 and under        949.
```

Write your answer here: The age group that has spent the most money on Legos is the age range 36-50. In total, in this dataset, they have spent 9,533 dollars on lego sets.

7. Which Lego theme has made the most money for Lego?

Hint: The `str_sub()` function extracts parts of text strings. Example: `str_sub("Star Wars", 1, 4)` returns `"Star"`. For this question, you'll need to: 1. Create a variable for total revenue per sale (quantity × us_price) 2. Group by theme 3. Sum the revenue But wait - the theme variable might have extra text after the main theme name. Use `str_sub(theme, 1, X)` to extract just the first X characters if needed. Experiment to find the right number!

```
# Your code here
    lego_sales %>%
      mutate(short_theme_name = str_sub(theme, 1, 8)) %>%
      group_by(short_theme_name) %>%
      summarise(total_revenue_per_sale = sum(us_price*quantity)) %>%
      arrange(desc(total_revenue_per_sale))
```

```
## # A tibble: 25 x 2
##    short_theme_name total_revenue_per_sale
##    <chr>                             <dbl>
```

```
## 1 Star War                         4448.
## 2 Ninjago                          2279.
## 3 City                             2211.
## 4 Nexo Kni                         2209.
## 5 Minecraf                         1550.
## 6 Gear                             1533.
## 7 Friends                          1279.
## 8 Duplo                            1220.
## 9 Elves                            1120.
## 10 Ghostbus                         880.
## # i 15 more rows
```

Write your answer here: The Star Wars Lego theme has made the most money for Lego in the dataset, with a total of 4448 dollars.

8. Which area code has spent the most money on Legos?

Hint: The `str_sub()` function extracts parts of text. For example: `str_sub("555-1234", 1, 3)` returns `"555"`. In the US, the area code is the first 3 digits of a phone number. You'll need to: 1. Use `mutate()` to create an `area_code` variable: `str_sub(phone_number, 1, 3)` 2. Create another variable for total spent per transaction 3. Group by the new area_code variable 4. Sum the total spent

```
# Your code here
  lego_sales %>%
    mutate(area_code = str_sub(phone_number, 1, 3)) %>%
    group_by(area_code) %>%
    summarise(total_spent_per_transaction = sum(us_price*quantity)) %>%
    arrange(desc(total_spent_per_transaction))
```

```
## # A tibble: 157 x 2
##    area_code total_spent_per_transaction
##    <chr>                           <dbl>
##  1 <NA>                            3993.
##  2 956                              720.
##  3 973                              685.
##  4 567                              550.
##  5 281                              465.
##  6 316                              438.
##  7 339                              426.
##  8 209                              350.
##  9 423                              340.
## 10 778                              335.
## # i 147 more rows
```

Write your answer here: The area code that has spent the most money on Legos is 956, with a total of 720 dollars. One interesting thing though is that the phone numbers who did not have an area code spent a total of 3993 dollars, given the summary table.

9. Come up with a question you want to answer using these data, and write it down. Then, create a data visualization that answers the question, and explain how your visualization answers the question.

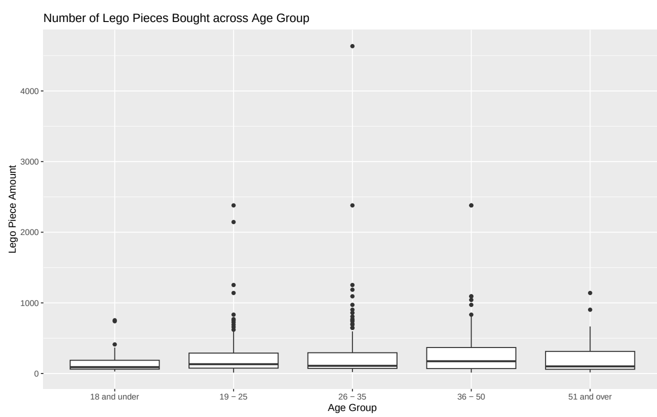
Be sure to include your research question, the variables you'll need to use to answer the question, the type of plot that will answer the question along with the code, and 2-3 sentences interpreting your results.

Research Question: Are different age groups more likely to purchase lego sets with more pieces? Variables Needed: I will need the age_group variable from Question 4 and the pieces variable that was given from the original data structure.

Type of Plot: I will use a box plot to answer this question so that I can compare quartile ranges and the median and maximums and minimums.

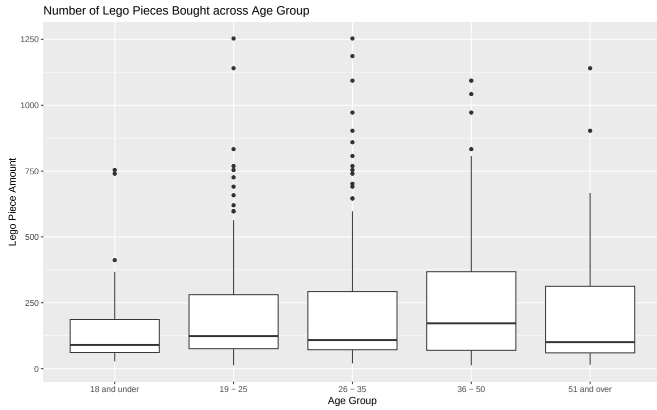
```
# Your code here
    lego_sales %>%
    ggplot(aes(x = age_group, y = pieces)) +
    geom_boxplot() +
    labs(
      title = "Number of Lego Pieces Bought across Age Group",
      x = "Age Group",
      y = "Lego Piece Amount"
    )
```

```
## Warning: Removed 69 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```




```
# Your code here
    lego_sales %>%
    filter(pieces<1500) %>%
    ggplot(aes(x = age_group, y = pieces)) +
    geom_boxplot() +
    labs(
      title = "Number of Lego Pieces Bought across Age Group",
      x = "Age Group",
```

```
  y = "Lego Piece Amount"
)
```



Yes, according to my box_plot, some age groups are more likely to buy sets with more lego pieces. More specifically, people in the ages 26-35 purchased the maximum number of lego pieces in the dataset, while people in the ages 36-50 have the highest median number of pieces purchased per set. I used the first graph to determine this. In comparison, those 18 and under have the lowest median number of pieces purchased per set. I used the second graph to determine this.

Knit, *commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards and review the md document on GitHub to make sure you're happy with the final state of your work.*

Now go back through your write up to make sure you've answered all questions and all of your R chunks are properly labeled.

Once you decide that you are done with the homework, choose the knit drop down and select `Knit to tufte_handout` to generate a pdf. Download and submit that pdf to Canvas.

Common Errors and Solutions

As you work through this assignment, you might encounter some errors. Here are the most common ones and how to fix them:

Error: "could not find function 'count'" or similar

- **Problem:** Packages aren't loaded
- **Solution:** Run `library(tidyverse)` and `library(dsbox)` in your Console

Error: "object 'lego_sales' not found"

- **Problem:** The dsbox package isn't loaded

- **Solution:** Run `library(dsbox)` in your Console

Error: “could not find function ‘%>%’ ”

- **Problem:** The tidyverse package isn’t loaded
- **Solution:** Run `library(tidyverse)` in your Console

Error: “object ‘______’ not found” (where ______ is a variable you created)

- **Problem:** You didn’t save your modified dataset or ran chunks out of order
- **Solution:** Make sure you used `<-` to save changes (e.g., `lego_sales <- lego_sales %>% mutate(...)`) and run chunks in order from top to bottom

Getting a lot of NA values in your results

- **Problem:** Some data might be missing
- **Solution:** Add `na.rm = TRUE` inside functions like `sum()`: `sum(quantity, na.rm = TRUE)`

Your pipe %>% isn’t working

- **Problem:** Check your syntax
- **Solution:** Make sure there’s a space before and after `%>%`, and that each new line of the pipeline is indented

Can’t find the answer you expected

- **Problem:** Double-check variable names (they’re case-sensitive!)
- **Solution:** Run `names(lego_sales)` to see all available variable names

Still stuck?

- Read the error message carefully - it often tells you what’s wrong
- Check your spelling and capitalization
- Make sure parentheses `()` and quotes `""` are matched
- Try running your code line by line to find where the error occurs
- Ask for help!